Simple Expense Tracker – Database Design (MySQL 8)

Oct 13, 2026 · @skr

1. Design principles and relationships

The database is **MySQL 8.0** (InnoDB), designed and tested in **MySQL Workbench** and run in production on **Amazon RDS for MySQL**. It has ten tables built around one central table: expenses. Each user owns their expenses, categories, limits and quick-add items.

Principles

- **Money is stored as whole paise** (₹10.50 = 1050) in a bigint, never as a float, so totals are always exact.
- **IDs are UUIDs created by the client**, so an entry made offline can be saved locally and synced later without clashes.
- **Soft delete.** Deleting sets `deleted_at`, which lets Undo work and lets other devices learn about the deletion during sync.
- **Every row has `updated_at`**, which drives sync and conflict resolution (latest write wins).
- **Reports are calculated from expenses**, not stored, so editing an entry can never leave a stale total.
- **Dates in the user's own day.** `spent_on` is a plain date in the user's time zone, so a ₹10 auto at 11:50 PM stays on the right day.

How the tables relate

Table	Belongs to	Relationship
users	–	One user has many expenses, limits, quick-add items, custom categories, devices
auth_otps	phone number	Short-lived codes for phone login; not linked to users until verified
refresh_tokens	users, devices	Many per user, one per signed-in device

Table	Belongs to	Relationship
devices	users	Push token and platform for reminders (phase 2)
categories	users (optional)	System categories have no owner; custom ones belong to a user
category_keywords	categories, users (optional)	Maps words like "auto" or "samosa" to a category; per-user rows hold learned corrections
expenses	users, categories	Many expenses per user; each has one category
quick_add_items	users, categories	The one-tap chips shown on the home screen
limits	users	At most one limit per user per period (week, month, year)
limit_alerts	limits	One row each time the 80% or 100% level is reached in a period

2. Table definitions (MySQL types)

Type choices. IDs are `CHAR(36)` UUIDs (easy to read in Workbench; use time-ordered UUIDv7 from the client so inserts stay fast in InnoDB). Timestamps are `DATETIME(3)` stored in UTC. Booleans are `TINYINT(1)`. Text uses `utf8mb4`, so Hindi and emoji work.

expenses (the core table)

Column	Type	Notes
id	CHAR(36), PK	Created by the client
user_id	CHAR(36), FK users	Owner
item_name	VARCHAR(100)	Text typed by the user, for example "Auto"
amount_paise	BIGINT	Greater than 0; ₹10 = 1000
currency	CHAR(3)	Default INR

Column	Type	Notes
category_id	CHAR(36), FK categories	Auto-assigned, editable
tag	ENUM('need','want')	Default need
spent_on	DATE	The user's local date
note	VARCHAR(250), null	Free text
created_at, updated_at	DATETIME(3)	updated_at is set by the API and drives sync
deleted_at	DATETIME(3), null	Soft delete

users

Column	Type	Notes
id	CHAR(36), PK	
phone	VARCHAR(15), unique, null	E.164 format, for OTP login
email	VARCHAR(255), unique, null	From Google sign-in
google_sub	VARCHAR(64), unique, null	Google account ID
display_name	VARCHAR(80), null	
language	VARCHAR(5)	en, hi; default en
timezone	VARCHAR(40)	Default Asia/Kolkata
week_start	TINYINT	1 = Monday (default), 0 = Sunday
reminder_enabled, reminder_time	TINYINT(1), TIME	Default off, 21:00:00
created_at, updated_at, deleted_at	DATETIME(3)	

categories and **category_keywords**

Column	Type	Notes
categories.id	CHAR(36), PK	
categories.user_id	CHAR(36), null	Null = system category shared by everyone
categories.name, name_hi	VARCHAR(40)	English and Hindi names
categories.icon	VARCHAR(30)	Icon key
categories.default_tag	ENUM('need','want')	Used when the user gives no tag
category_keywords.id	CHAR(36), PK	
category_keywords.category_id	CHAR(36), FK	
category_keywords.user_id	CHAR(36), null	Null = system keyword; set = learned from this user's correction
category_keywords.keyword	VARCHAR(50)	For example "auto", "samosa", "recharge"; the default collation ignores case

System categories at launch: Food, Travel, Bills, Shopping, Health, Education, Entertainment, Groceries, Other.

quick_add_items

Column	Type	Notes
id	CHAR(36), PK	
user_id	CHAR(36), FK	
item_name	VARCHAR(100)	Unique per user (case-insensitive by collation)
default_amount_paise	BIGINT	Last used amount
category_id, tag	CHAR(36) FK, ENUM	Reused when the chip is tapped

Column	Type	Notes
use_count	INT	Used to rank chips
last_used_at	DATETIME(3)	
is_pinned	TINYINT(1)	Pinned chips always show

limits and **limit_alerts**

Column	Type	Notes
limits.id	CHAR(36), PK	
limits.user_id	CHAR(36), FK	
limits.period	ENUM('week','month','year')	One row per user per period
limits.amount_paise	BIGINT	Greater than 0
limits.is_active	TINYINT(1)	Lets users switch a limit off without losing it
limit_alerts.id	CHAR(36), PK	
limit_alerts.limit_id	CHAR(36), FK	
limit_alerts.period_start	DATE	First day of the week, month or year
limit_alerts.level	TINYINT	80 or 100
limit_alerts.triggered_at	DATETIME(3)	Unique on (limit_id, period_start, level) so each alert fires once

Authentication tables

Table	Columns
auth_otps	id, phone, code_hash, expires_at (5 minutes), attempts, consumed_at
refresh_tokens	id, user_id, device_id, token_hash, expires_at (30 days), revoked_at, created_at

Table	Columns
devices	id, user_id, platform (web, android, ios), push_token, app_version, last_seen_at

3. MySQL schema (run in Workbench)

Requires MySQL 8.0.16 or later (CHECK constraints are enforced from that version, and `DEFAULT (UUID())` needs 8.0.13). Amazon RDS for MySQL 8.0 supports all of it.

```
CREATE DATABASE IF NOT EXISTS expense_tracker
  CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE expense_tracker;

CREATE TABLE users (
  id               CHAR(36)     NOT NULL DEFAULT (UUID()),
  phone            VARCHAR(15)  NULL,
  email            VARCHAR(255) NULL,
  google_sub       VARCHAR(64)  NULL,
  display_name     VARCHAR(80)  NULL,
  language         VARCHAR(5)   NOT NULL DEFAULT 'en',
  timezone         VARCHAR(40)  NOT NULL DEFAULT 'Asia/Kolkata',
  week_start       TINYINT      NOT NULL DEFAULT 1,
  reminder_enabled TINYINT(1)   NOT NULL DEFAULT 0,
  reminder_time    TIME         NOT NULL DEFAULT '21:00:00',
  created_at       DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at       DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  deleted_at       DATETIME(3)  NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_users_phone (phone),
  UNIQUE KEY uq_users_email (email),
  UNIQUE KEY uq_users_google (google_sub),
  CONSTRAINT chk_users_week_start CHECK (week_start IN (0, 1)),
  CONSTRAINT chk_users_login CHECK (phone IS NOT NULL OR email IS NOT NULL OR
google_sub IS NOT NULL)
) ENGINE=InnoDB;

CREATE TABLE devices (
  id           CHAR(36)     NOT NULL DEFAULT (UUID()),
  user_id      CHAR(36)     NOT NULL,
  platform     ENUM('web','android','ios') NOT NULL,
```

```
  push_token   VARCHAR(512) NULL,
  app_version  VARCHAR(20)  NULL,
  last_seen_at DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  KEY idx_devices_user (user_id),
  CONSTRAINT fk_devices_user FOREIGN KEY (user_id) REFERENCES users (id) ON
DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE refresh_tokens (
  id         CHAR(36)    NOT NULL DEFAULT (UUID()),
  user_id    CHAR(36)    NOT NULL,
  device_id  CHAR(36)    NULL,
  token_hash CHAR(64)    NOT NULL,
  expires_at DATETIME(3) NOT NULL,
  revoked_at DATETIME(3) NULL,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uq_refresh_hash (token_hash),
  KEY idx_refresh_user (user_id),
  CONSTRAINT fk_refresh_user   FOREIGN KEY (user_id)   REFERENCES users (id)
ON DELETE CASCADE,
  CONSTRAINT fk_refresh_device FOREIGN KEY (device_id) REFERENCES devices
(id) ON DELETE SET NULL
) ENGINE=InnoDB;

CREATE TABLE auth_otps (
  id          CHAR(36)    NOT NULL DEFAULT (UUID()),
  phone       VARCHAR(15) NOT NULL,
  code_hash   CHAR(64)    NOT NULL,
  expires_at  DATETIME(3) NOT NULL,
  attempts    TINYINT UNSIGNED NOT NULL DEFAULT 0,
  consumed_at DATETIME(3) NULL,
  created_at  DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  KEY idx_otps_phone (phone, created_at)
) ENGINE=InnoDB;

CREATE TABLE categories (
  id          CHAR(36)    NOT NULL DEFAULT (UUID()),
  user_id     CHAR(36)    NULL,                    -- NULL = system category
  name        VARCHAR(40) NOT NULL,
  name_hi     VARCHAR(40) NULL,
```

```
  icon        VARCHAR(30) NULL,
  default_tag ENUM('need','want') NOT NULL DEFAULT 'need',
  created_at  DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  deleted_at  DATETIME(3) NULL,
  PRIMARY KEY (id),
  KEY idx_categories_user (user_id),
  CONSTRAINT fk_categories_user FOREIGN KEY (user_id) REFERENCES users (id)
ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE category_keywords (
  id          CHAR(36)    NOT NULL DEFAULT (UUID()),
  category_id CHAR(36)    NOT NULL,
  user_id     CHAR(36)    NULL,                    -- NULL = system keyword
(managed by the seed script)
  keyword     VARCHAR(50) NOT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_keyword (user_id, keyword),
  KEY idx_keyword_lookup (keyword),
  CONSTRAINT fk_kw_category FOREIGN KEY (category_id) REFERENCES categories
(id) ON DELETE CASCADE,
  CONSTRAINT fk_kw_user     FOREIGN KEY (user_id)     REFERENCES users (id)
ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE expenses (
  id           CHAR(36)     NOT NULL,              -- generated by the client
(UUIDv7)
  user_id      CHAR(36)     NOT NULL,
  item_name    VARCHAR(100) NOT NULL,
  amount_paise BIGINT       NOT NULL,
  currency     CHAR(3)      NOT NULL DEFAULT 'INR',
  category_id  CHAR(36)     NOT NULL,
  tag          ENUM('need','want') NOT NULL DEFAULT 'need',
  spent_on     DATE         NOT NULL,
  note         VARCHAR(250) NULL,
  created_at   DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at   DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  deleted_at   DATETIME(3)  NULL,
  PRIMARY KEY (id),
  KEY idx_expenses_user_date (user_id, deleted_at, spent_on),
  KEY idx_expenses_user_cat  (user_id, category_id, spent_on),
```

```
  KEY idx_expenses_sync      (user_id, updated_at),
  CONSTRAINT chk_expenses_amount CHECK (amount_paise > 0),
  CONSTRAINT fk_expenses_user     FOREIGN KEY (user_id)     REFERENCES users
(id) ON DELETE CASCADE,
  CONSTRAINT fk_expenses_category FOREIGN KEY (category_id) REFERENCES
categories (id)
) ENGINE=InnoDB;

CREATE TABLE quick_add_items (
  id                   CHAR(36)     NOT NULL DEFAULT (UUID()),
  user_id              CHAR(36)     NOT NULL,
  item_name            VARCHAR(100) NOT NULL,
  default_amount_paise BIGINT       NOT NULL,
  category_id          CHAR(36)     NULL,
  tag                  ENUM('need','want') NOT NULL DEFAULT 'need',
  use_count            INT          NOT NULL DEFAULT 1,
  last_used_at         DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  is_pinned            TINYINT(1)   NOT NULL DEFAULT 0,
  PRIMARY KEY (id),
  UNIQUE KEY uq_quick_user_item (user_id, item_name),
  CONSTRAINT chk_quick_amount CHECK (default_amount_paise > 0),
  CONSTRAINT fk_quick_user     FOREIGN KEY (user_id)     REFERENCES users
(id) ON DELETE CASCADE,
  CONSTRAINT fk_quick_category FOREIGN KEY (category_id) REFERENCES
categories (id)
) ENGINE=InnoDB;

CREATE TABLE limits (
  id           CHAR(36)    NOT NULL DEFAULT (UUID()),
  user_id      CHAR(36)    NOT NULL,
  period       ENUM('week','month','year') NOT NULL,
  amount_paise BIGINT      NOT NULL,
  is_active    TINYINT(1)  NOT NULL DEFAULT 1,
  created_at   DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at   DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uq_limits_user_period (user_id, period),
  CONSTRAINT chk_limits_amount CHECK (amount_paise > 0),
  CONSTRAINT fk_limits_user FOREIGN KEY (user_id) REFERENCES users (id) ON
DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE limit_alerts (
```

```
  id           CHAR(36)    NOT NULL DEFAULT (UUID()),
  limit_id     CHAR(36)    NOT NULL,
  period_start DATE        NOT NULL,
  level        TINYINT     NOT NULL,
  triggered_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (id),
  UNIQUE KEY uq_alert (limit_id, period_start, level),
  CONSTRAINT chk_alert_level CHECK (level IN (80, 100)),
  CONSTRAINT fk_alert_limit FOREIGN KEY (limit_id) REFERENCES limits (id) ON
DELETE CASCADE
) ENGINE=InnoDB;
```

Seed data (run once after the schema)

```
INSERT INTO categories (user_id, name, name_hi, icon, default_tag) VALUES
  (NULL, 'Food',          'खाना',      'food',          'need'),
  (NULL, 'Travel',        'यात्रा',    'travel',        'need'),
  (NULL, 'Bills',         'बिल',       'bills',         'need'),
  (NULL, 'Shopping',      'खरीदारी',   'shopping',      'want'),
  (NULL, 'Health',        'स्वास्थ्य', 'health',        'need'),
  (NULL, 'Education',     'शिक्षा',    'education',     'need'),
  (NULL, 'Entertainment', 'मनोरंजन',   'entertainment', 'want'),
  (NULL, 'Groceries',     'किराना',    'groceries',     'need'),
  (NULL, 'Other',         'अन्य',      'other',         'need');

INSERT INTO category_keywords (category_id, user_id, keyword)
SELECT c.id, NULL, k.keyword
FROM (
  SELECT 'Travel' AS cat, 'auto' AS keyword UNION ALL SELECT 'Travel', 'bus'
  UNION ALL SELECT 'Travel', 'metro'       UNION ALL SELECT 'Travel',
'petrol'
  UNION ALL SELECT 'Food', 'tea'           UNION ALL SELECT 'Food', 'samosa'
  UNION ALL SELECT 'Food', 'lunch'         UNION ALL SELECT 'Food', 'dinner'
  UNION ALL SELECT 'Food', 'snacks'
  UNION ALL SELECT 'Bills', 'recharge'     UNION ALL SELECT 'Bills',
'electricity'
  UNION ALL SELECT 'Bills', 'wifi'
  UNION ALL SELECT 'Groceries', 'vegetables' UNION ALL SELECT 'Groceries',
'milk'
  UNION ALL SELECT 'Groceries', 'rice'
) k
JOIN categories c ON c.name = k.cat AND c.user_id IS NULL;
```

Using MySQL Workbench

1. Open a connection to your local MySQL 8 server, then File > New Query Tab, paste the schema and run it with the lightning button (Ctrl+Shift+Enter). Run the seed script after it.
2. To see the diagram, choose Database > Reverse Engineer, pick `expense_tracker`, and Workbench builds an EER diagram. Save it as a `.mwb` model file in the repository.
3. To change the design visually, edit the EER model, then use Database > Forward Engineer or Synchronize Model to produce the SQL.
4. Keep Workbench's Safe Updates option on (Edit > Preferences > SQL Editor) so a mistyped DELETE without a WHERE clause is blocked.
5. For production, do not edit tables by hand. Save every schema change as a numbered migration file (Flyway, Prisma Migrate or TypeORM migrations) and let the deployment run it. Workbench is then used only to look at data and run queries.
6. To open the AWS database in Workbench, use the connection method Standard TCP/IP over SSH (through a bastion host) or an AWS Systems Manager port-forward. Never make the production database publicly reachable.

4. Report and limit queries

All reports run directly on `expenses` and use the index `idx_expenses_user_date`. In Workbench, set the parameters first; the API passes the same three values from code (the first and last date of the period, worked out from the user's time zone and week start). Every query ignores soft-deleted rows.

```
SET @user_id = '<a user id>';
SET @from    = '2026-10-12';
SET @to      = '2026-10-18';

-- Total and count for a period (week, month or year)
SELECT COALESCE(SUM(amount_paise), 0) AS total_paise, COUNT(*) AS entries
FROM expenses
WHERE user_id = @user_id AND deleted_at IS NULL
  AND spent_on BETWEEN @from AND @to;

-- Spending by category (dashboard chart 1, biggest category in the report)
SELECT c.id, c.name, SUM(e.amount_paise) AS total_paise
FROM expenses e JOIN categories c ON c.id = e.category_id
WHERE e.user_id = @user_id AND e.deleted_at IS NULL
```

```
  AND e.spent_on BETWEEN @from AND @to
GROUP BY c.id, c.name
ORDER BY total_paise DESC;

-- Spending per day (dashboard chart 2)
SELECT spent_on, SUM(amount_paise) AS total_paise
FROM expenses
WHERE user_id = @user_id AND deleted_at IS NULL
  AND spent_on BETWEEN @from AND @to
GROUP BY spent_on
ORDER BY spent_on;

-- Need vs Want split
SELECT tag, SUM(amount_paise) AS total_paise
FROM expenses
WHERE user_id = @user_id AND deleted_at IS NULL
  AND spent_on BETWEEN @from AND @to
GROUP BY tag;

-- Top 3 single expenses
SELECT id, item_name, amount_paise, spent_on
FROM expenses
WHERE user_id = @user_id AND deleted_at IS NULL
  AND spent_on BETWEEN @from AND @to
ORDER BY amount_paise DESC
LIMIT 3;

-- Top items by total (for example, "Tea: 14 times, ₹280")
SELECT LOWER(item_name) AS item, COUNT(*) AS times, SUM(amount_paise) AS
total_paise
FROM expenses
WHERE user_id = @user_id AND deleted_at IS NULL
  AND spent_on BETWEEN @from AND @to
GROUP BY item
ORDER BY total_paise DESC
LIMIT 5;

-- Year view: totals per month (replaces the daily chart for range = year)
SELECT DATE_FORMAT(spent_on, '%Y-%m') AS month, SUM(amount_paise) AS
total_paise
FROM expenses
WHERE user_id = @user_id AND deleted_at IS NULL
  AND spent_on BETWEEN @from AND @to
```

```
GROUP BY month
ORDER BY month;

-- Record a limit alert safely (does nothing if it already exists)
INSERT IGNORE INTO limit_alerts (limit_id, period_start, level)
VALUES ('<limit id>', '2026-10-12', 80);
-- ROW_COUNT() = 1 means the alert is new, so show it to the user
```

Change vs last week. Run the first query twice, once for this week and once for the previous week, and calculate the difference and percentage in the API.

Limit progress. For each active limit, run the total query for the current period and calculate:

percent used = spent in period ÷ limit amount × 100

When percent used first reaches 80 or 100 in a period, insert a row into `limit_alerts`. The unique key makes the insert safe to repeat, and the API returns the new alert to the client only when the insert succeeds.

Period boundaries. Week = from the user's week start day to six days later; month = first to last calendar day; year = 1 January to 31 December. (Switch to an April–March year if users ask for a financial year.)

5. Offline sync, privacy and growth

Local database. The web app uses IndexedDB and the mobile apps use SQLite. The local tables mirror `expenses`, `categories`, `quick_add_items` and `limits`, plus two extra columns on each row: `sync_state` (synced, pending, deleted_pending) and `server_updated_at`.

Sync flow

1. Guest users work only on the local database. No server calls are needed.
2. On sign-in, the app uploads all local rows to the server in one batch. Because IDs are client-generated UUIDs, the upload is safe to repeat.
3. After that, each change is saved locally first and marked pending. The app pushes pending rows when it is online.
4. The app asks the server for everything changed after its last sync cursor (the server's `updated_at` of the newest row it received) and applies the changes.

5. If the same row was changed on two devices, the row with the newer `updated_at` wins. Deleted rows stay as soft-deleted rows for 90 days so every device can learn about them.

Privacy and security

- Store only what is needed: phone or email, expenses, limits. No bank data, no contacts.
- OTPs and refresh tokens are stored as SHA-256 hashes, never as plain text.
- Account deletion removes the user row, and `ON DELETE CASCADE` removes expenses, limits, chips and tokens. A scheduled job also purges soft-deleted rows older than 90 days.
- On AWS, turn on RDS storage encryption (KMS) and automated backups, and keep the database in the Mumbai region (ap-south-1) so data stays in India.
- The app connects with its own MySQL user that has only the rights it needs (no DROP or CREATE); migrations run with a separate admin user.
- Do not log item names or amounts in application logs.

Growth and migrations

- A single RDS MySQL instance handles millions of expense rows with the indexes above. If one user logs 10 expenses a day, a million users produce about 3.7 billion rows a year, so plan to partition `expenses` by year (MySQL RANGE partitioning on `spent_on`) or archive old years once the table passes roughly 100 million rows. Partitioned InnoDB tables cannot have foreign keys, so decide on this before the table grows, and enforce `user_id` and `category_id` in the API at that point.
- Add an RDS read replica for reports when report queries slow down the main database.
- Use time-ordered UUIDv7 IDs from the client. Random UUIDv4 values scatter inserts across the primary-key index and slow InnoDB down as the table grows.
- Likely future columns: `payment_mode` (cash, UPI, card) on expenses, a `recurring_rules` table for P2 recurring expenses, `currency` on limits when multi-currency is added.